Recommended Test Application

OpenTelemetry Astronomy Shop Demo

Repository: github.com/open-telemetry/opentelemetry-demo | **Docs:** opentelemetry.io/docs/demo/

Decision

Use the **OpenTelemetry Astronomy Shop Demo** as the primary testbed for the AI SRE Copilot. Start with its native feature-flag scenarios, then add Grafana LGTM and Jaeger through deployment configuration.

Why this is the right choice

- Real distributed behavior across HTTP, gRPC, Kafka, PostgreSQL, and Valkey.
- First-class OpenTelemetry traces, metrics, logs, exemplars, dashboards, and telemetry tests.
- Official architecture, service, telemetry, feature-flag, and debugging documentation for RAG.
- Native, reversible fault injection through OpenFeature and flagd.
- Faults cover downstream failures, latency, database contention, queues, resource leaks, and AI loops.
- Most integration work is configuration and a Copilot telemetry adapter, not application rewrites.

Observability model

`Astronomy Shop services → OpenTelemetry Collector → Prometheus / Tempo / Jaeger / Loki → Grafana`

The upstream deployment already includes the application, Collector, Prometheus, Grafana, and Jaeger/OpenSearch paths. Add Loki and Tempo with a small Collector overlay or Grafana's `otel-lgtm` backend. Retain Jaeger as a second trace backend if required.

Native fault scenarios

Scenario	What it tests
paymentFailure	Repeatable known incident and memory recall
paymentUnreachable	Same symptom, different downstream cause
intlShippingSlowdown	Cohort-specific latency
productCatalogLockContention	Database contention and cascading latency
kafkaQueueProblems	Async backpressure and consumer lag
recommendationCacheFailure	Gradual memory and latency degradation
aiRunawayAgent	AI/tool-loop failure diagnosis

Fit for the AI SRE Copilot

- Prometheus threshold breach starts proactive analysis.
- Trace and log correlation identifies the failing service or dependency.
- Official documentation explains expected behavior and diagnostic signals.
- Hindsight recalls prior incidents and successful fixes.
- A repeated fault can use fast memory verification; a changed context can reject stale memory.

Recommended rollout

1. Start with `paymentFailure`, `paymentUnreachable`, and `intlShippingSlowdown`. 2. Connect read-only Prometheus, Loki, Tempo, and Jaeger tools. 3. Index official demo documentation. 4. Add Kafka and database-lock scenarios. 5. Use Chaos Mesh or Istio later for infrastructure-level experiments.

Conclusion

Astronomy Shop is the strongest practical choice because it combines realistic distributed behavior, maintained OpenTelemetry support, rich documentation, deterministic native faults, and a low-risk integration path. It is more capable than a small Petclinic-style app without the operational burden of a large SaaS platform such as PostHog.

Prepared as a concise architecture decision brief for the AI SRE Copilot testbed.